

南開大學
Nankai University

计算机学院

计算机组成原理 lab2 实验报告

乘法器的优化

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 4 月 16 日

目录

1 实验目标 **2**

2 实验原理 **2**
2.1 指导书参考乘法器原理 2
2.2 顶层模块框图说明 2
2.3 基础优化版原理图说明 3
2.4 华莱士树优化版原理图说明 3

3 基础优化版设计与实现 **4**
3.1 基础版 multiply.v 模块的修改 4
3.2 基础版仿真文件 testbench.v 的编写与测试 5
3.3 lcd_module 模块的导入 5
3.4 顶层模块 multiply_display.v 的改写 5
3.5 约束文件 multiply.xdc 的配置 6
3.6 基础版综合、实现与上板流程 6

4 基础优化版实验结果 **6**
4.1 基础版波形仿真结果 6
4.2 基础版波形数据分析 7
4.3 基础版实验箱上板结果 8
4.4 基础版实验箱结果分析 10

5 华莱士树优化版设计与实现 **10**
5.1 华莱士树版 multiply_wallace.v 的实现 10
5.2 华莱士树版仿真文件的修改与测试 11
5.3 华莱士树版接入顶层模块 11
5.4 华莱士树版综合、实现与上板流程 11

6 华莱士树优化版实验结果 **12**
6.1 华莱士树版波形仿真结果 12
6.2 华莱士树版波形数据分析 13
6.3 华莱士树版实验箱上板结果 13
6.4 华莱士树版实验箱结果分析 15

7 两种优化方案性能对比分析 **15**

8 实验收获 **17**

A 附录 **18**

1 实验目标

本实验围绕“数据运算：定点乘法”展开。实验指导书中给出的要求主要包括：理解定点乘法的不同实现算法，重点掌握迭代乘法的实现方法；使用 Verilog 完成乘法器设计；完成仿真、外围模块设计以及 FPGA 上板验证。

结合本次实验的实际内容，实验目标如下：

1. 理解定点乘法器的基本实现原理，掌握有符号补码乘法的处理方法；
2. 熟悉 Vivado 和 Verilog 的基本使用，能够完成乘法器模块的设计、仿真与调试；
3. 在指导书参考设计基础上，对原始迭代乘法器进行改进，实现每个时钟周期移动两位的基础优化版乘法器，提高乘法效率；
4. 在完成基础优化版的基础上，进一步尝试华莱士树乘法器实现，并分析其性能改进效果；
5. 完成外围显示模块设计与约束配置，使设计能够在实验箱上输入两个乘数并显示乘法结果；
6. 通过波形仿真和实验箱上板结果验证设计的正确性，并对不同优化方案的时钟周期进行比较分析。

2 实验原理

2.1 指导书参考乘法器原理

指导书中的参考设计采用的是迭代乘法算法。该算法实现的是有符号乘法，但在具体运算时，先对两个输入数取绝对值参与运算，得到的乘积也是绝对值，最后再根据两个输入数的符号位对结果进行校正。也就是说，参考设计采用的是“绝对值运算 + 符号恢复”的实现方式。

迭代乘法的过程与手工二进制乘法类似。每个时钟周期中，乘数右移一位，被乘数左移一位，再根据乘数当前最低位的值决定本拍是否把被乘数累加到乘积寄存器中。若最低位为 1，则加上当前被乘数；若最低位为 0，则加 0。这样不断移位并累加，最终得到乘法结果。

由于该算法本质上是通过多次加法完成一次乘法，因此需要多个时钟周期才能完成运算。指导书中给出的参考设计对于 32 位乘法，最长需要 32 个时钟周期才能完成一次运算。这也是后续进行性能优化的出发点。

2.2 顶层模块框图说明

本实验采用顶层模块 `multiply_display.v` 作为外围显示模块。该模块负责调用乘法器模块和 `lcd_module`，通过实验箱触摸屏输入两个乘数，并在 LCD 上显示两个输入数和最终乘法结果。

其中，`mult_op1` 和 `mult_op2` 用于保存两个输入乘数，`sw_begin` 用于启动乘法运算，`led_end` 用于指示乘法结束。乘法器输出的 64 位乘积被分为高 32 位和低 32 位，分别显示为 `PRO_H` 和 `PRO_L`。由于乘法运算需要多个时钟周期，因此只有当 `mult_end` 有效时，结果才真正有效，此时再将结果锁存后显示到 LCD 上。

2.3 基础优化版原理图说明

基础优化版是在指导书参考迭代乘法器原理图的基础上修改得到的，整体仍然保留“先取绝对值，最后恢复符号”的实现思路。与指导书参考版相比，基础优化版的主要改动是将原来每拍处理 1 位乘数改为每拍处理 2 位乘数，因此被乘数寄存器每拍左移两位，乘数寄存器每拍右移两位。

同时，部分积生成逻辑也由原来的二选一改为四选一。基础优化版根据 `multiplier[1:0]` 的四种取值生成不同部分积：当最低两位为 `00` 时加 0，为 `01` 时加 1 倍被乘数，为 `10` 时加 2 倍被乘数，为 `11` 时加 3 倍被乘数。然后将当前部分积与 `product_temp` 累加，最终得到乘法结果。

因此，基础优化版的核心思想是在不改变总体结构的前提下，通过提高每拍处理位数来减少整体迭代次数。对于 32 位乘法，该方法理论上可将最长周期由 32 个时钟周期降低到 16 个时钟周期。

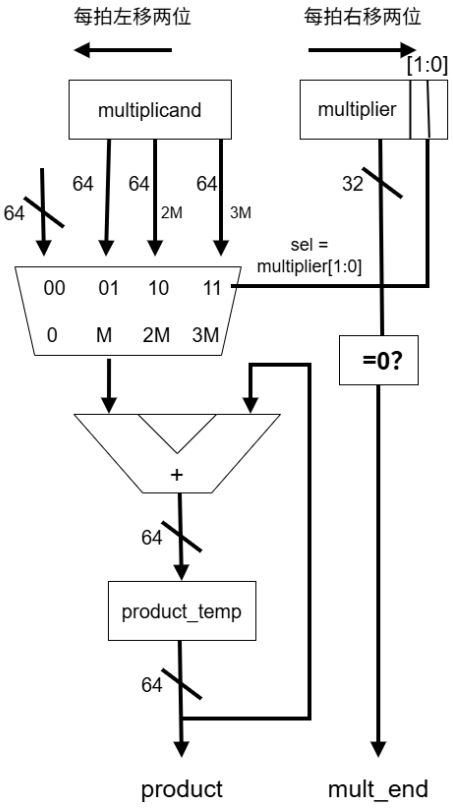


图 2.1: 基础优化版乘法器原理图

2.4 华莱士树优化版原理图说明

在完成基础优化版之后，本实验进一步实现了华莱士树优化版乘法器。该版本不再采用逐拍移位累加的方式，而是先对两个输入数取绝对值，再并行生成全部部分积，随后使用多级 CSA 压缩树将多路部分积逐级压缩，最后通过一次普通加法得到无符号乘积。

在本实验的实现中，并行部分积由 `pp[0:31]` 表示，压缩过程从 `stage1` 一直到 `stage7`，最终得到两路压缩结果 `wallace_a` 和 `wallace_b`，再通过最终加法器得到 `unsigned_product`。之后将其锁存到 `product_temp` 中，并根据 `op1_sign ^ op2_sign` 恢复补码形式的输出结果。

另外，为了兼容原有顶层模块，本实验中增加了 `mult_valid` 和 `mult_end_r` 两个控制信号，使乘法结束信号延后一拍输出。这样可以使外围显示模块更稳定地锁存结果。相比基础优化版，华莱士树优化版不再依赖逐拍移位和逐拍累加，而是采用并行部分积生成与压缩树归约的方式，因此具有更高的运算效率。

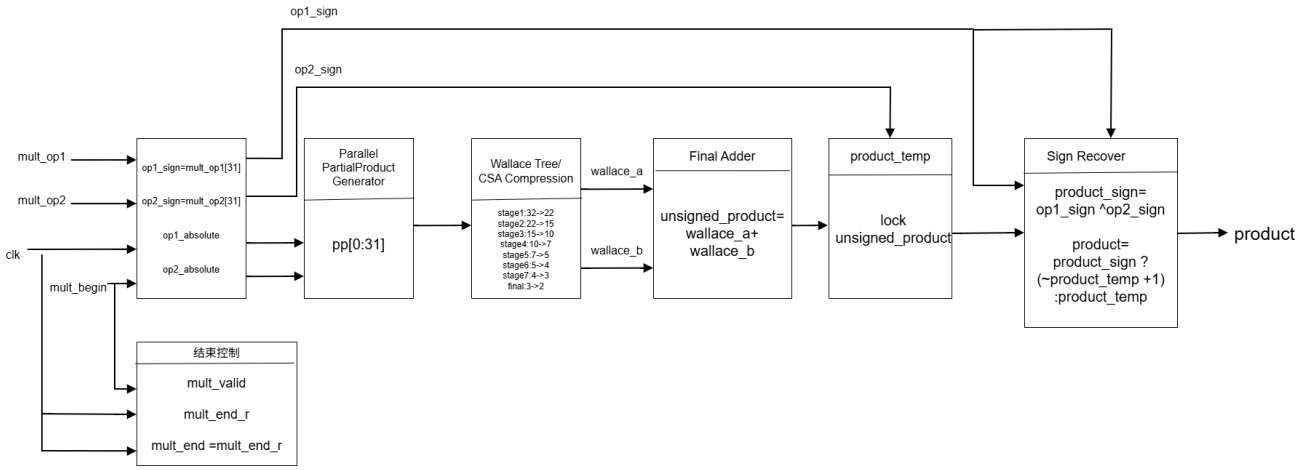


图 2.2: 华莱士树优化版乘法器原理图

3 基础优化版设计与实现

3.1 基础版 multiply.v 模块的修改

基础优化版在指导书参考乘法器的基础上进行修改，整体结构没有改变，仍然保留了乘法开始信号 `mult_begin`、运算中标志 `mult_valid`、结束信号 `mult_end` 以及“先取绝对值，最后恢复符号”的实现方式。

本次修改的核心是把原来每个时钟只处理 1 位乘数，改成每个时钟处理 2 位乘数。具体来说，被乘数寄存器 `multiplicand` 原来每拍左移 1 位，现在改成每拍左移 2 位；乘数寄存器 `multiplier` 原来每拍右移 1 位，现在改成每拍右移 2 位。对应代码如下：

```
multiplicand <= {multiplicand[61:0],2'b00};
multiplier   <= {2'b00,multiplier[31:2]};
```

由于现在每拍处理的是乘数最低两位，所以部分积生成逻辑也要修改。原来只需要判断 `multiplier[0]` 是 0 还是 1，现在需要判断 `multiplier[1:0]` 的四种情况：00、01、10、11。因此本实验使用 `case` 语句生成四种部分积，分别对应加 0、加 1 倍被乘数、加 2 倍被乘数和加 3 倍被乘数。代码如下：

```
always @(*) begin
    case (multiplier[1:0])
        2'b00: partial_product = 64'd0;
        2'b01: partial_product = multiplicand;
        2'b10: partial_product = (multiplicand << 1);
        2'b11: partial_product = multiplicand + (multiplicand << 1);
        default: partial_product = 64'd0;
    endcase
end
```

累加器部分仍然保持原来的思路不变，只是每拍累加新的两位部分积：

```
product_temp <= product_temp + partial_product;
```

最后，结果符号仍然由两个输入数的符号位异或得到，若结果应为负，则对绝对值乘积取反加一：

```
assign product = product_sign ? (~product_temp+1) : product_temp;
```

通过上述修改，乘法器从原来的“1 位乘”改成了“2 位乘”，理论上最长周期由 32 个时钟周期减少为 16 个时钟周期。

3.2 基础版仿真文件 testbench.v 的编写与测试

为了验证基础优化版乘法器的正确性，本实验编写了独立的 testbench.v 文件。测试文件中首先产生周期为 10ns 的时钟信号：

```
initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end
```

为了方便测试不同数据，仿真文件中定义了任务 run_case。该任务的功能是：输入一组乘数，拉高 mult_begin 启动乘法，等待 mult_end 置位，同时统计乘法完成所经历的时钟周期数，并将结果与理论值进行比较。任务中使用 cycle_cnt 对周期数进行统计，代码如下：

```
while (!mult_end) begin
    @(posedge clk);
    cycle_cnt = cycle_cnt + 1;
end
```

本实验共测试了三组数据：13 × 11、-13 × 11 和 -25 × -4。这样可以分别验证正数乘正数、负数乘正数和负数乘负数三种情况。仿真结果表明，基础优化版乘法器在三组测试数据下都能够得到正确结果，且样例中统计得到的周期数为 4。

3.3 lcd_module 模块的导入

为了在实验箱上输入两个乘数并显示乘法结果，本实验直接导入指导书提供的 lcd_module 模块。该模块本身不需要修改，只需要在顶层模块中正确实例化即可。

在本实验中，lcd_module 的作用主要有两个：一是接收触摸屏输入的数据，并通过 input_valid 和 input_value 输出给顶层模块；二是根据 display_number、display_name 和 display_value 在 LCD 上显示当前需要展示的内容。因此，lcd_module 主要负责输入输出界面，而真正的乘法计算仍然由 multiply.v 完成。

3.4 顶层模块 multiply_display.v 的改写

顶层模块 multiply_display.v 的作用是连接乘法器模块和实验箱外设。在基础优化版中，顶层实例化的乘法器应为 multiply 模块。开始信号直接由开关 sw_begin 提供，因此有：

```
assign mult_begin = sw_begin;
assign led_end = mult_end;
```

其中，led_end 用于在板上显示乘法是否结束。

顶层模块中还定义了两个寄存器 mult_op1 和 mult_op2，分别保存两个输入乘数。通过 input_sel 选择当前写入的是哪一个乘数：当 input_sel=0 时，将触摸屏输入值写入 mult_op1；当 input_sel=1 时，将输入值写入 mult_op2。对应代码如下：

```
else if (input_valid && !input_sel)
    mult_op1 <= input_value;

else if (input_valid && input_sel)
    mult_op2 <= input_value;
```

由于乘法结果不是立刻有效的，而是在若干拍之后由 mult_end 指示完成，因此顶层中使用寄存器 product_r 对结果进行锁存。只有当 mult_end 有效时，才把 product 写入 product_r，再用于显示。这样可以避免在乘法过程中把中间结果直接显示到 LCD 上。

显示部分共设置了四个显示项，分别为 M_OP1、M_OP2、PRO_H 和 PRO_L，对应两个输入乘数和 64 位乘积的高低 32 位。这样在实验箱上可以直接观察输入数据和乘法结果。

3.5 约束文件 multiply.xdc 的配置

约束文件 multiply.xdc 用于完成顶层端口与实验箱实际引脚之间的连接。本实验中主要配置了系统时钟 clk、复位信号 resetn、输入选择开关 input_sel、乘法开始开关 sw_begin、结束指示灯 led_end，以及 LCD 触摸屏相关引脚。

在配置过程中，LCD 相关引脚直接沿用指导书给出的约束，不需要修改。只要保证 multiply_display.v 中的端口名与 multiply.xdc 中的约束名称一致，就可以正常完成综合、实现和上板。

3.6 基础版综合、实现与上板流程

完成代码后，先将 multiply.v、multiply_display.v、lcd_module.dcp 和 multiply.xdc 加入 Vivado 工程，并将 multiply_display.v 设为顶层模块。之后依次执行综合、实现和 bit 文件生成。

生成 bit 文件后，打开 Hardware Manager，将 bit 文件下载到 FPGA 实验箱。下载成功后，可以通过触摸屏输入两个乘数。输入时先用 input_sel 选择当前输入的是 M_OP1 还是 M_OP2，再点击屏幕输入具体数据；输入完成后，拉高 sw_begin 启动乘法运算。运算结束后，LCD 上显示 PRO_H 和 PRO_L，同时 led_end 亮起，表示乘法完成。通过这种方式即可完成基础优化版乘法器的上板验证。

4 基础优化版实验结果

4.1 基础版波形仿真结果

基础优化版共进行了三组仿真测试，分别为正数乘正数、负数乘正数和负数乘负数。图 4.3、图 4.4 和图 4.5 给出了对应的仿真波形。

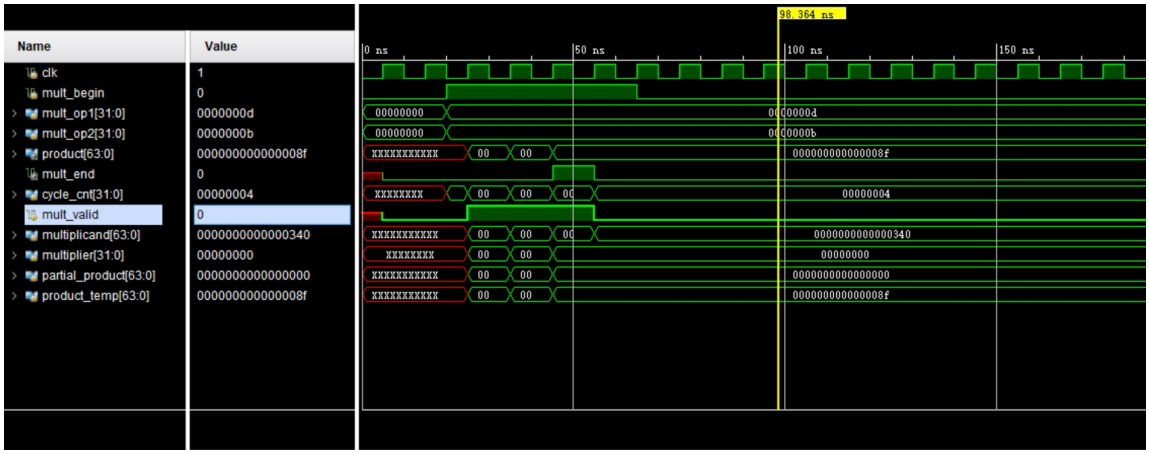


图 4.3: 基础优化版波形仿真结果（13×11）

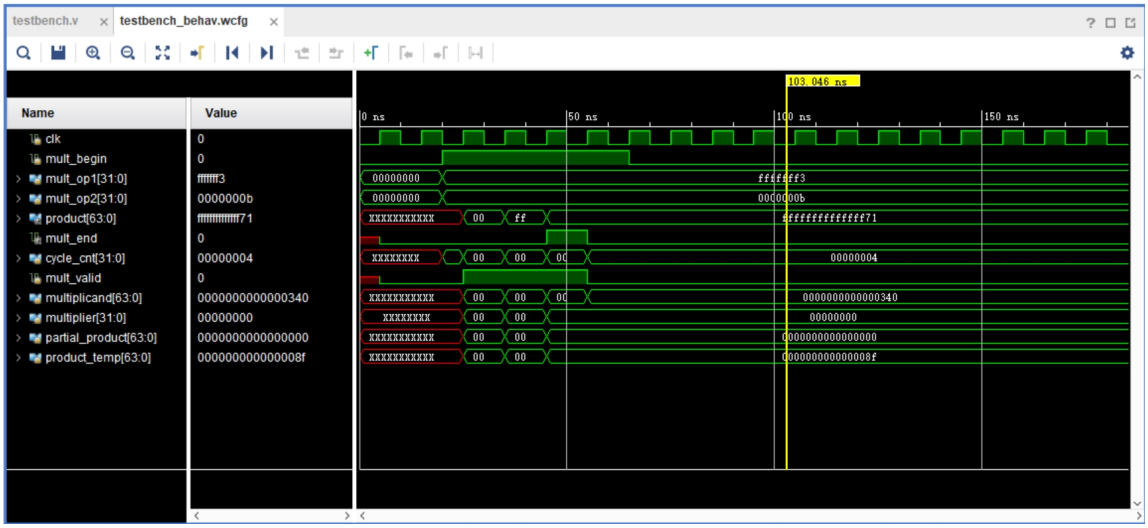


图 4.4: 基础优化版波形仿真结果（-13×11）

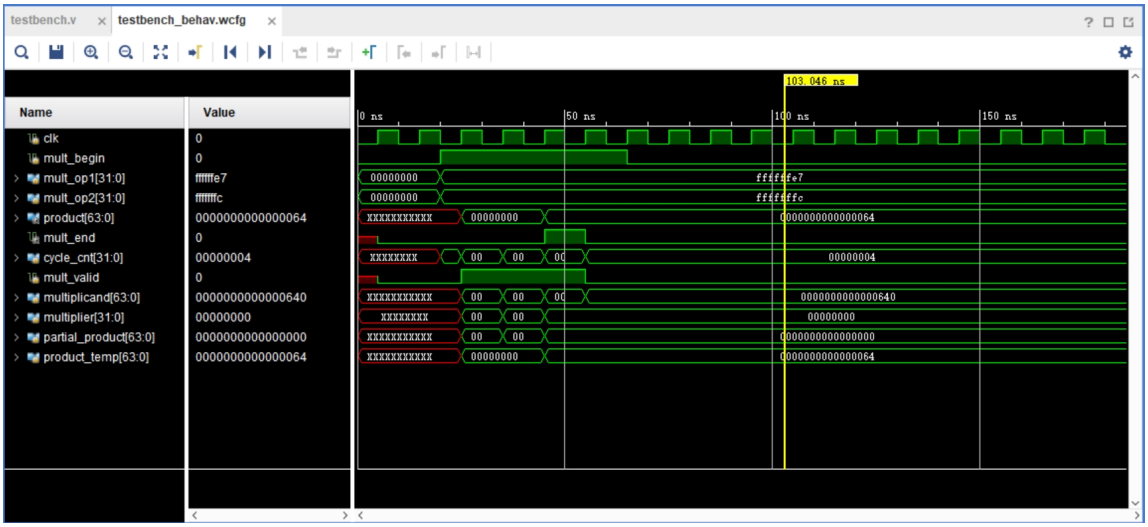


图 4.5: 基础优化版波形仿真结果（-25×-4）

从仿真结果可以看出，基础优化版乘法器能够正确输出三组测试数据的乘法结果，说明修改后的乘法器逻辑正确。

4.2 基础版波形数据分析

第一组仿真输入为 13 × 11。波形中可见 mult_op1=0000000D，mult_op2=0000000B，最终输出 product=000000000000008F，对应十进制结果为 143，结果正确。

第二组仿真输入为 -13 × 11。波形中可见 mult_op1=FFFFFFF3，mult_op2=0000000B，最终输出 product=FFFFFFFFFFFFFF71，对应十进制结果为 -143，结果正确。这说明乘法器能够正确处理负数输入，并完成符号恢复。

第三组仿真输入为 -25 × -4。波形中可见 mult_op1=FFFFFFE7，mult_op2=FFFFFFFC，最终输出 product=0000000000000064，对应十进制结果为 100，结果正确。这说明当两个输入都为负数时，乘法器也能够正确得到正结果。

另外，从三张波形图中都可以看到 `cycle_cnt=4`。这说明在本次测试样例下，基础优化版乘法器只用了 4 个时钟周期就完成了乘法运算。与指导书原始设计最长 32 个时钟周期相比，基础优化版通过每拍处理 2 位乘数，明显降低了运算周期。

4.3 基础版实验箱上板结果

完成仿真后，将基础优化版下载到 FPGA 实验箱中进行上板验证。图 4.6、图 4.7 和图 4.8 给出了三组测试数据在实验箱上的显示结果。

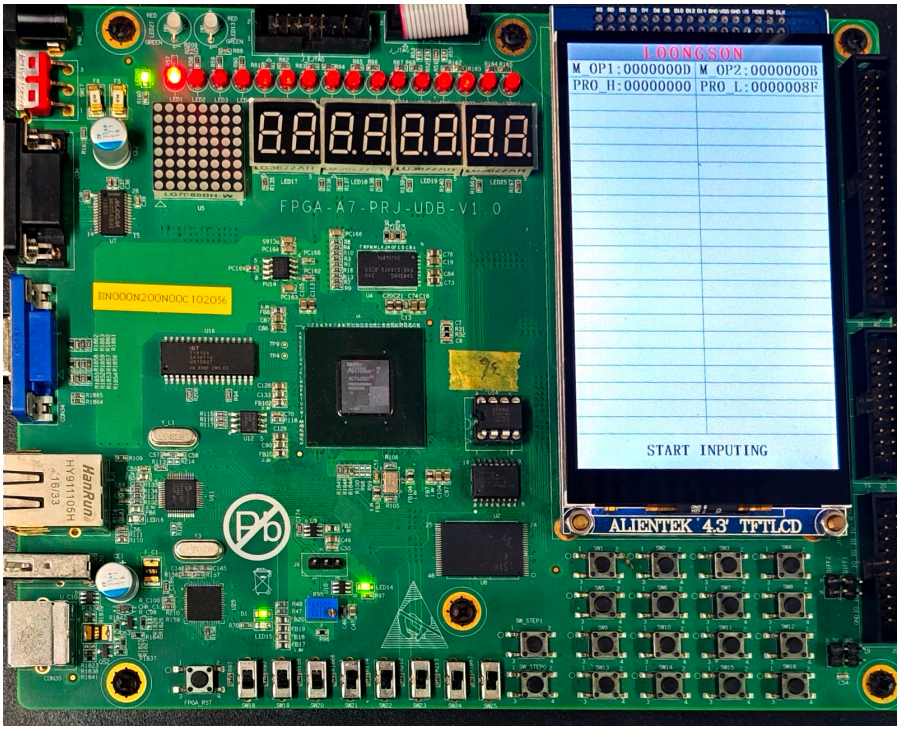


图 4.6: 基础优化版实验箱上板结果（13×11）

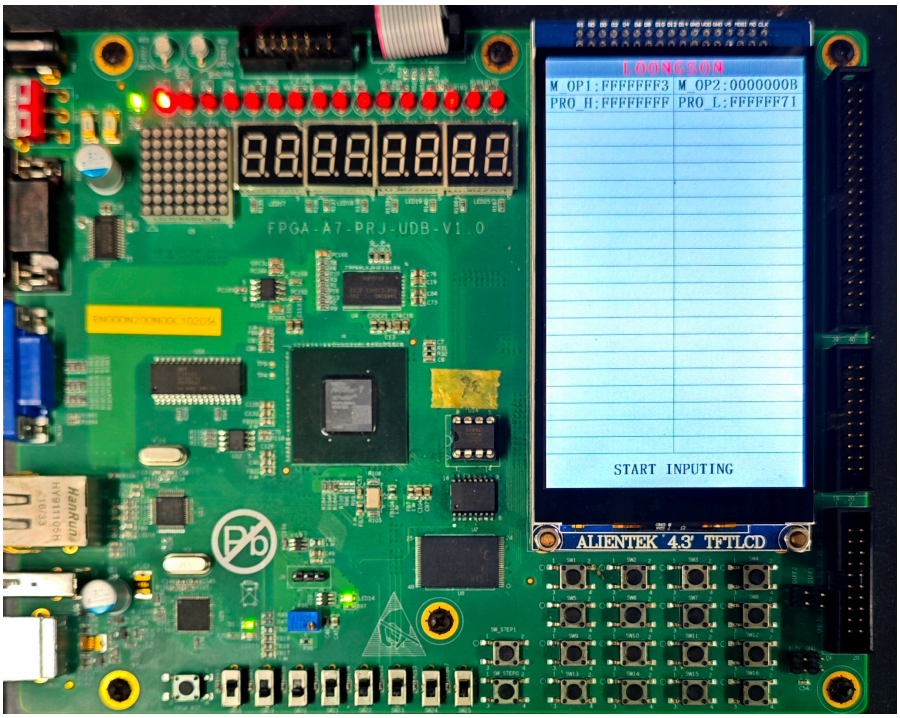


图 4.7: 基础优化版实验箱上板结果（-13×11）

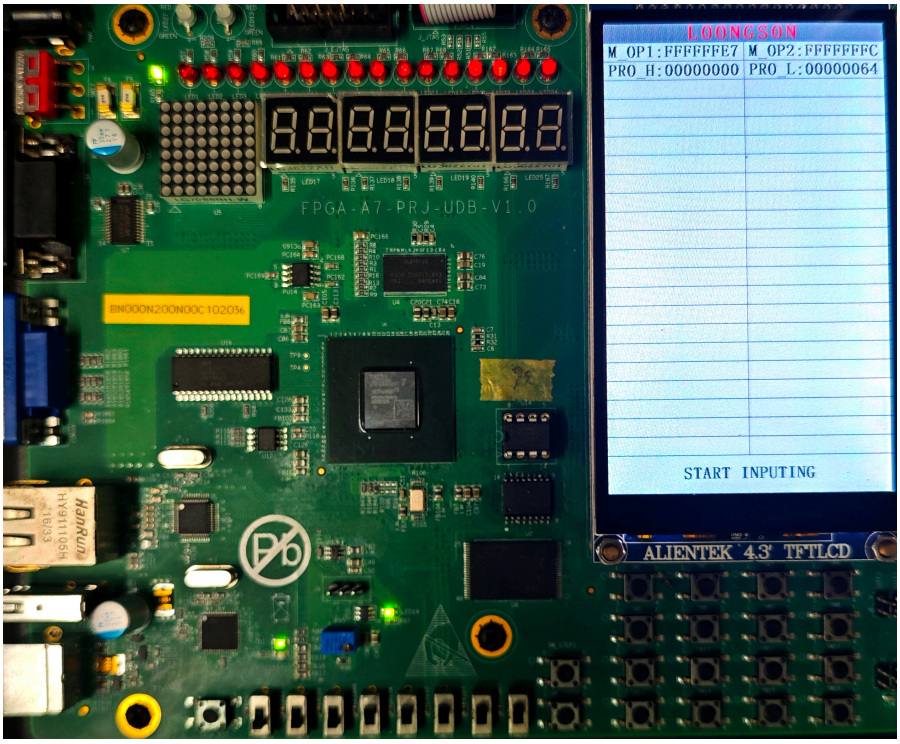


图 4.8: 基础优化版实验箱上板结果（-25×-4）

从上板结果可以看出，基础优化版乘法器能够在实验箱上正确完成输入、运算和显示，板上结果与仿真结果一致。

4.4 基础版实验箱结果分析

对于第一组 13 × 11，实验箱上显示 M_OP1=0000000D，M_OP2=0000000B，结果为 PRO_H=00000000，PRO_L=0000008F。合并后得到 64 位结果 0x000000000000008F，即十进制 143，结果正确。

对于第二组 -13 × 11，实验箱上显示 M_OP1=FFFFFFF3，M_OP2=0000000B，结果为 PRO_H=FFFFFFFF，PRO_L=FFFFFF71。合并后得到 64 位结果 0xFFFFFFFFFFFFFF71，即十进制 -143，结果正确。

对于第三组 -25 × -4，实验箱上显示 M_OP1=FFFFFFE7，M_OP2=FFFFFFFC，结果为 PRO_H=00000000，PRO_L=00000064。合并后得到 64 位结果 0x0000000000000064，即十进制 100，结果正确。

综合仿真结果和实验箱结果可以说明，基础优化版乘法器能够正确完成 32 位有符号乘法运算，功能正确，优化有效。

5 华莱士树优化版设计与实现

5.1 华莱士树版 multiply_wallace.v 的实现

在完成基础优化版之后，本实验进一步实现了华莱士树优化版乘法器 multiply_wallace.v。该版本仍然保留了指导书参考设计的接口形式，即输入为 clk、mult_begin、mult_op1、mult_op2，输出为 product 和 mult_end，这样可以方便后续直接接入原有顶层模块。

华莱士树版的主要思路是不再采用逐拍移位累加的迭代方式，而是先并行生成全部部分积，再利用多级压缩树逐步把多个操作数压缩成两个操作数，最后再进行一次普通加法，得到无符号乘积。代码中首先对两个输入数取绝对值：

```
assign op1_absolute = op1_sign ? (~mult_op1 + 32'd1) : mult_op1;
assign op2_absolute = op2_sign ? (~mult_op2 + 32'd1) : mult_op2;
```

之后使用 for 循环一次性生成 32 个部分积：

```
for (i = 0; i < 32; i = i + 1) begin
    if (op2_absolute[i]) begin
        pp[i] = ({32'd0, op1_absolute} << i);
    end
    else begin
        pp[i] = 64'd0;
    end
end
```

为了实现压缩树，代码中定义了两个 CSA 函数 csa_sum 和 csa_carry，分别用于把 3 个操作数压缩成 2 个操作数中的和与进位。随后通过 stage1 到 stage7 多级压缩，把 32 路部分积逐步压缩到 2 路，最后计算：

```
unsigned_product = wallace_a + wallace_b;
```

得到绝对值乘积后，再根据两个输入符号位异或得到结果符号，并恢复最终补码输出：

```
assign product = product_sign ? (~product_temp + 64'd1) : product_temp;
```

由于华莱士树的主体是并行组合逻辑，为了兼容原有显示模块，本实验没有直接把 mult_end 做成组合结束信号，而是增加了 mult_valid 和 mult_end_r，将结束信号延后一拍输出。这样在实验箱显示时，外围模块可以更稳定地锁存结果。

5.2 华莱士树版仿真文件的修改与测试

华莱士树版仍然使用与基础版相同的仿真框架，只是在 testbench.v 中把被测模块由 multiply 改成了 multiply_wallace：

```
multiply_wallace uut (
    .clk       (clk),
    .mult_begin(mult_begin),
    .mult_op1  (mult_op1),
    .mult_op2  (mult_op2),
    .product   (product),
    .mult_end  (mult_end)
);
```

时钟产生方式、run_case 任务以及周期统计方法都与基础版相同，仍然通过 cycle_cnt 统计从 mult_begin 拉高到 mult_end 有效之间所经历的时钟周期数。测试数据仍然选取三组典型情况，即 13 × 11、-13 × 11 和 -25 × -4，分别用于验证正数乘法、负数乘法以及负数乘负数三种情况。

从仿真结果来看，华莱士树版在三组测试数据下都能够得到正确乘积，并且样例中统计得到的周期数为 3。与基础优化版样例中的 4 个周期相比，华莱士树版周期进一步减少，说明该优化方案具有更高的效率。

5.3 华莱士树版接入顶层模块

为了实现上板验证，需要将华莱士树版乘法器接入顶层模块 multiply_display.v。基础版顶层中原来实例化的是 multiply，在华莱士树版中改为实例化 multiply_wallace：

```
multiply_wallace multiply_module (
    .clk       (clk       ),
    .mult_begin(mult_begin),
    .mult_op1  (mult_op1  ),
    .mult_op2  (mult_op2  ),
    .product   (product   ),
    .mult_end  (mult_end  )
);
```

其余输入输出逻辑基本保持不变。顶层中仍然通过 input_sel 选择当前输入的是 mult_op1 还是 mult_op2，通过 sw_begin 产生乘法开始信号，通过 led_end 指示乘法结束。由于乘法结果需要在 mult_end 有效时才能稳定显示，因此顶层中继续使用 product_r 对结果进行锁存：

```
else if (mult_end)
begin
    product_r <= product;
end
```

显示部分仍然在 LCD 上输出 M_OP1、M_OP2、PRO_H 和 PRO_L 四项信息，因此华莱士树版在接入顶层后，实验箱的输入和显示方式与基础版保持一致，只是内部乘法器模块发生了替换。

5.4 华莱士树版综合、实现与上板流程

华莱士树版的综合、实现与上板流程与基础版基本一致。首先将 multiply_wallace.v、multiply_display.v、lcd_module.dcp 和 multiply.xdc 加入工程，并将 multiply_display.v 设为顶层模块。然后依次执行综合、实现和 bit 文件生成。

生成 bit 文件后，通过 Vivado 的 Hardware Manager 将其下载到 FPGA 实验箱。下载完成后，仍然通过触摸屏输入两个乘数，使用 input_sel 选择写入 M_OP1 或 M_OP2，再通过 sw_begin 启动乘法运算。运算结束后，LCD 上显示 PRO_H 和 PRO_L，同时 led_end 给出结束指示。

由于华莱士树版的外部接口与基础版保持一致，因此在上板操作流程上没有额外变化，只需要把顶层调用的乘法器模块替换成 `multiply_wallace` 即可完成验证。

6 华莱士树优化版实验结果

6.1 华莱士树版波形仿真结果

华莱士树优化版同样进行了三组仿真测试，分别为正数乘正数、负数乘正数和负数乘负数。图 6.9、图 6.10 和图 6.11 给出了对应的仿真波形。

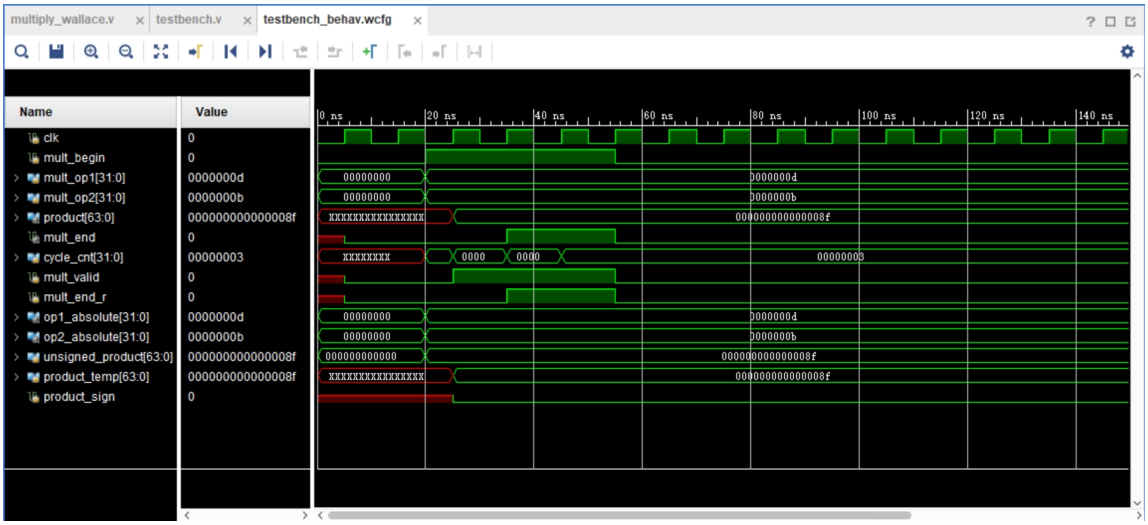


图 6.9: 华莱士树优化版波形仿真结果（13×11）

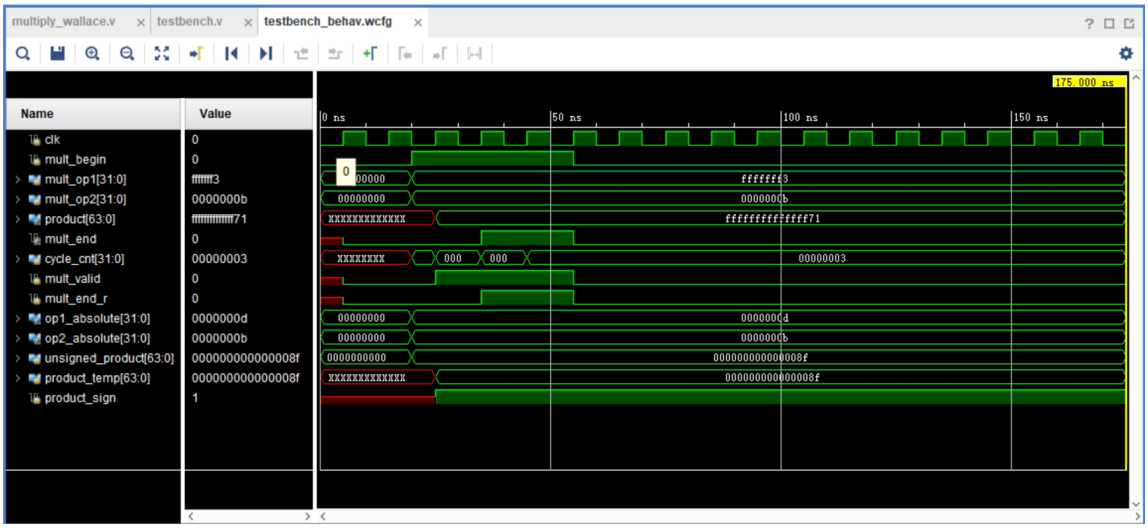


图 6.10: 华莱士树优化版波形仿真结果（-13×11）

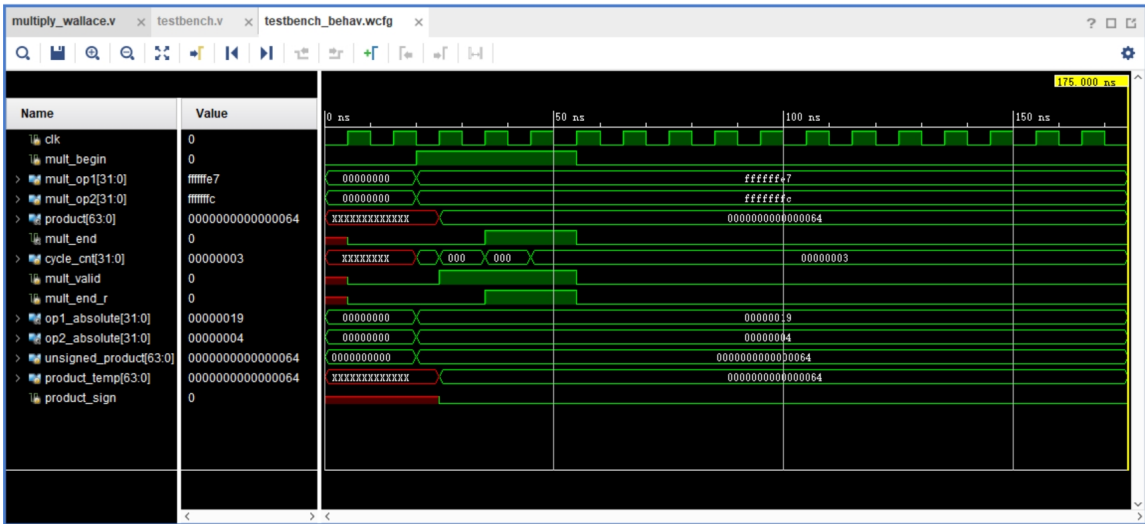


图 6.11: 华莱士树优化版波形仿真结果（-25×-4）

从仿真结果可以看出，华莱士树优化版乘法器能够正确输出三组测试数据的乘法结果，说明该版本乘法器逻辑正确。

6.2 华莱士树版波形数据分析

第一组仿真输入为 `13 × 11`。波形中可见 `mult_op1=0000000D`，`mult_op2=0000000B`，最终输出 `product=000000000000008F`，对应十进制结果为 143，结果正确。

第二组仿真输入为 `-13 × 11`。波形中可见 `mult_op1=FFFFFFF3`，`mult_op2=0000000B`，最终输出 `product=FFFFFFFFFFFFFF71`，对应十进制结果为 -143，结果正确。这说明华莱士树优化版乘法器能够正确处理负数输入，并完成符号恢复。

第三组仿真输入为 `-25 × -4`。波形中可见 `mult_op1=FFFFFFE7`，`mult_op2=FFFFFFFC`，最终输出 `product=0000000000000064`，对应十进制结果为 100，结果正确。这说明当两个输入都为负数时，华莱士树优化版乘法器也能够正确得到正结果。

另外，从三张波形图中都可以看到 `cycle_cnt=3`。这说明在本次测试样例下，华莱士树优化版乘法器只用了 3 个时钟周期就完成了乘法运算。相比指导书原始设计最长 32 个时钟周期，以及基础优化版样例中的 4 个时钟周期，华莱士树优化版进一步缩短了运算时间，说明该优化方案具有更高的效率。

6.3 华莱士树版实验箱上板结果

完成仿真后，将华莱士树优化版下载到 FPGA 实验箱中进行上板验证。图 6.12、图 6.13 和图 6.14 给出了三组测试数据在实验箱上的显示结果。

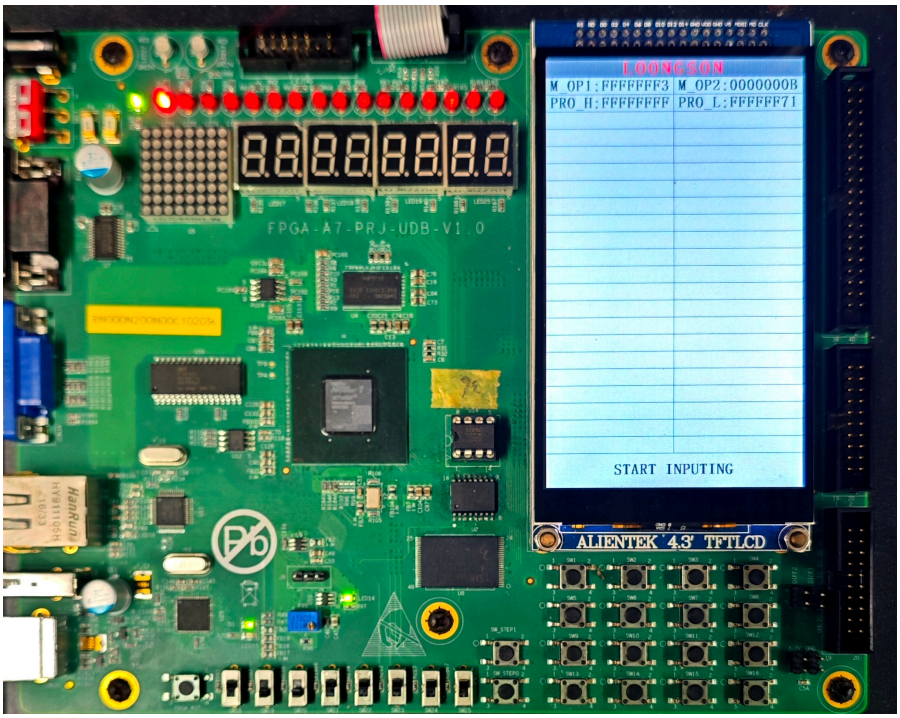


图 6.12: 华莱士树优化版实验箱上板结果（13×11）

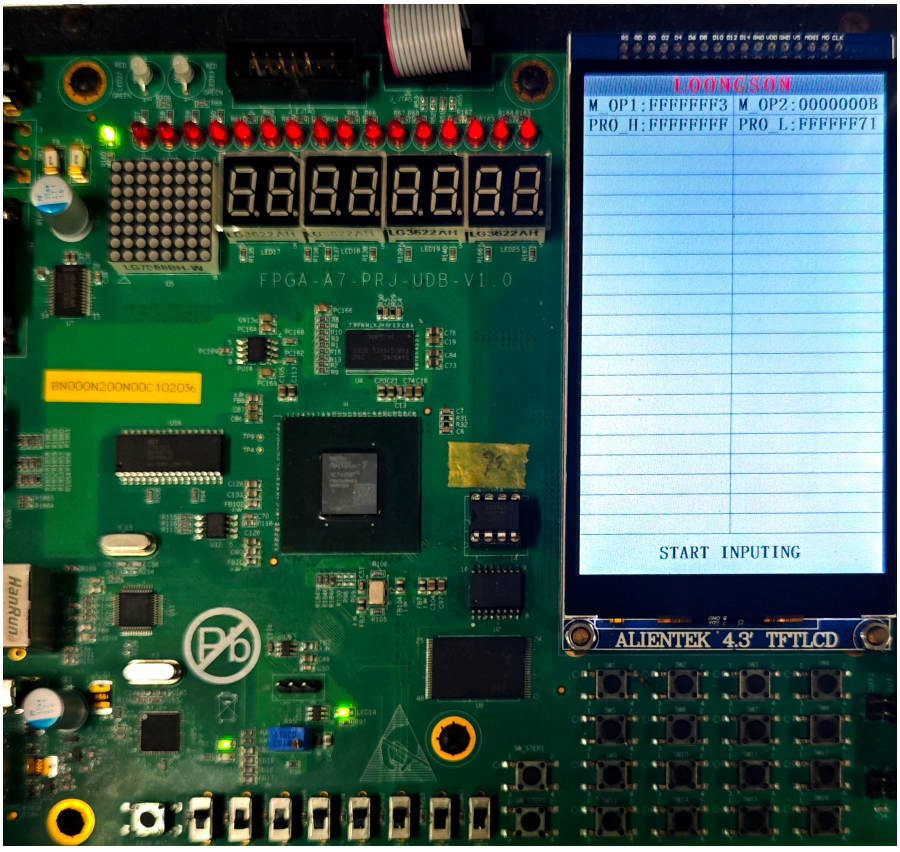


图 6.13: 华莱士树优化版实验箱上板结果（-13×11）

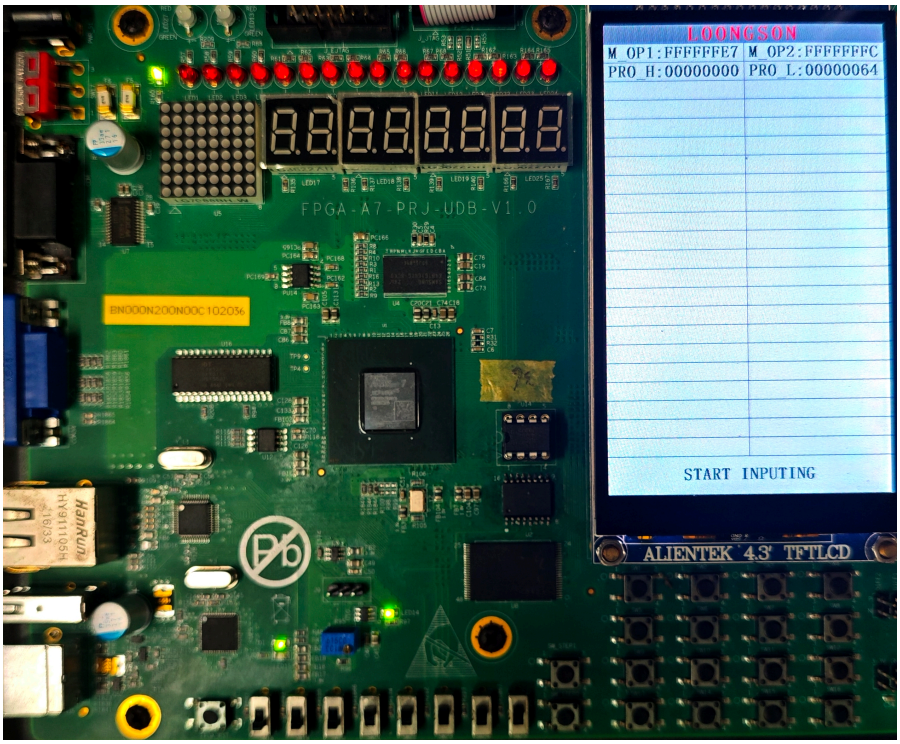


图 6.14: 华莱士树优化版实验箱上板结果（-25×-4）

从上板结果可以看出，华莱士树优化版乘法器能够在实验箱上正确完成输入、运算和显示，板上结果与仿真结果一致。

6.4 华莱士树版实验箱结果分析

对于第一组 13 × 11，实验箱上显示 M_OP1=0000000D，M_OP2=0000000B，结果为 PRO_H=00000000，PRO_L=0000008F。合并后得到 64 位结果 0x000000000000008F，即十进制 143，结果正确。

对于第二组 -13 × 11，实验箱上显示 M_OP1=FFFFFFF3，M_OP2=0000000B，结果为 PRO_H=FFFFFFFF，PRO_L=FFFFFF71。合并后得到 64 位结果 0xFFFFFFFFFFFFFF71，即十进制 -143，结果正确。

对于第三组 -25 × -4，实验箱上显示 M_OP1=FFFFFFE7，M_OP2=FFFFFFFC，结果为 PRO_H=00000000，PRO_L=00000064。合并后得到 64 位结果 0x0000000000000064，即十进制 100，结果正确。

综合仿真结果和实验箱结果可以说明，华莱士树优化版乘法器能够正确完成 32 位有符号乘法运算，并且相比基础优化版具有更短的运算周期，达到了进一步优化的目的。

7 两种优化方案性能对比分析

为了更加直观地比较基础优化版和华莱士树优化版三种乘法器的性能，我测试了指导书参考版在三组相同测试数据下的仿真波形。测试数据分别为 13 × 11、-13 × 11 和 -25 × -4，对应波形如图 7.15、图 7.16 和图 7.17 所示。

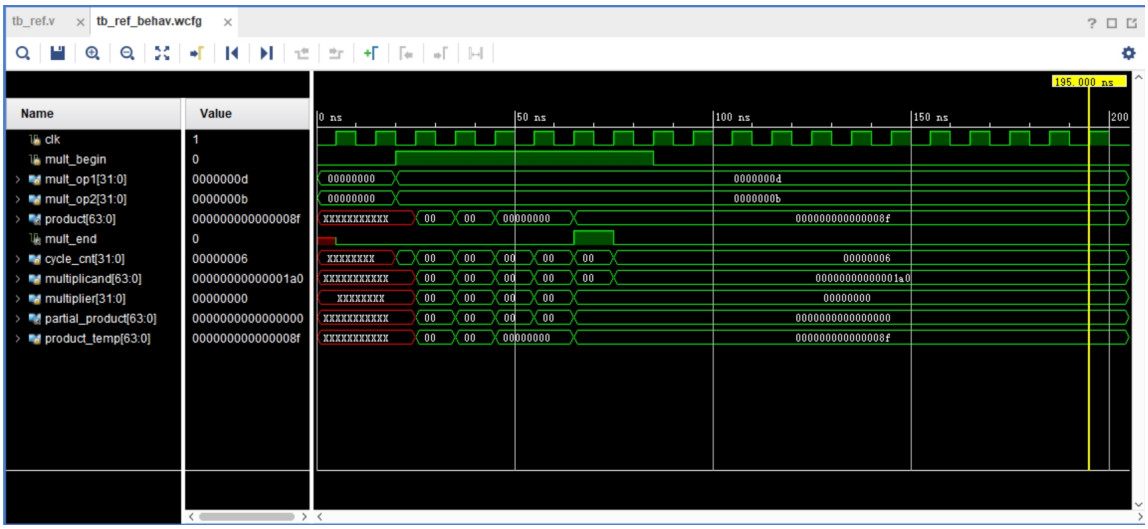


图 7.15: 指导书参考版仿真波形（13×11）

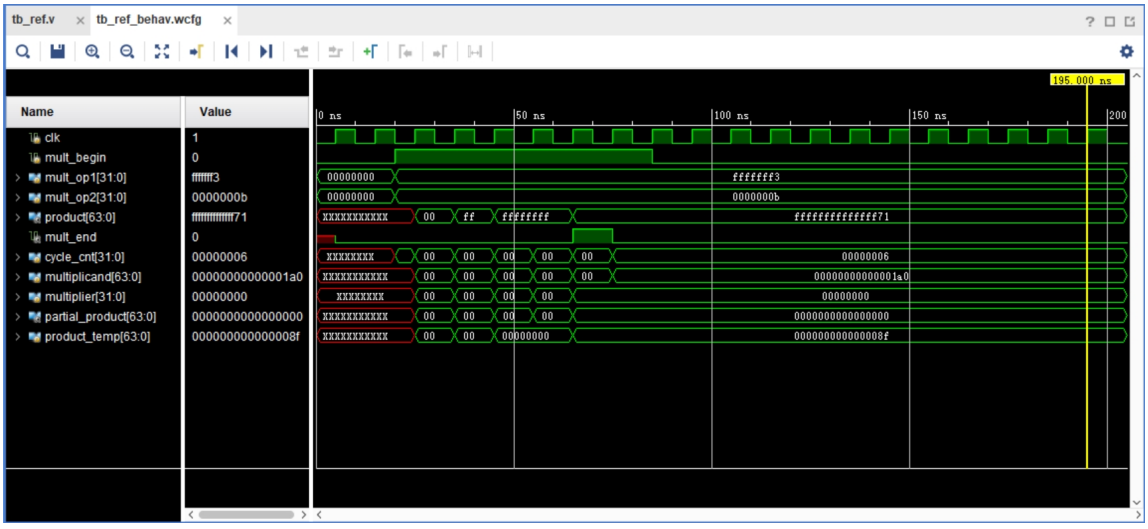


图 7.16: 指导书参考版仿真波形（-13×11）

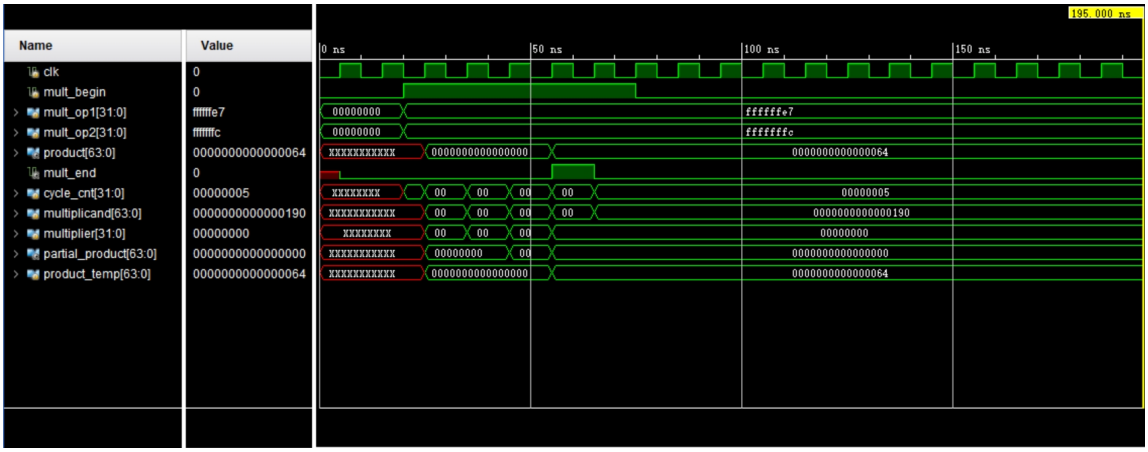


图 7.17: 指导书参考版仿真波形（-25×-4）

从上述波形可以看出，指导书参考版能够正确完成三组测试数据的乘法运算。其中，`13 × 11` 和 `-13 × 11` 两组样例的仿真周期数均为 6，`-25 × -4` 这组样例的仿真周期数为 5。虽然原始设计功能正确，但由于采用的是逐位迭代乘法，因此运算周期相对较长。

本实验在指导书参考乘法器的基础上，分别实现了基础优化版和华莱士树优化版。指导书参考版采用的是 1 位迭代乘法，即每个时钟周期处理乘数的 1 位，因此对于 32 位乘法，理论上最长需要 32 个时钟周期才能完成一次运算。基础优化版将每拍处理的位数由 1 位提高到 2 位，因此理论上最长周期可降为 16 个时钟周期。华莱士树优化版则不再采用逐拍移位累加的迭代方式，而是通过并行部分积生成和压缩树归约完成乘法运算，因此其性能进一步提高。

为了保证比较公平，三种方案均采用相同的三组测试数据进行仿真，并统一按照 testbench 中从 `mult_begin` 拉高到 `mult_end` 有效之间所经历的时钟周期数统计样例周期。结合本次实验的仿真结果，可以得到表 1 所示的性能对比。

表 1: 三种方案性能对比

方案	实现方式	理论最长周期	仿真样例周期		
			13×11	-13×11	-25×-4
指导书参考版	1 位迭代乘法	32	6	6	5
基础优化版	2 位迭代乘法	16	4	4	4
华莱士树优化版	并行部分积 + 压缩树	–	3	3	3

其中，表中的仿真样例周期依次对应三组测试数据：`13 × 11`、`-13 × 11` 和 `-25 × -4`。可以看出，在相同测试条件下，指导书参考版的样例周期分别为 6、6、5，基础优化版均为 4，而华莱士树优化版均为 3，说明优化后乘法器的执行效率得到了明显提升。

从表中可以看出，基础优化版相比指导书参考版已经有了明显改进。其主要方法是在保持原有迭代结构基本不变的前提下，将每个时钟周期处理的乘数位数由 1 位提高到 2 位，从而减少了整体迭代次数。虽然本次仿真样例中统计得到的周期数均为 4，但从原理上看，其最坏情况下的周期数已经由 32 降低到了 16。

华莱士树优化版则进一步提高了乘法器的运算效率。该版本通过并行生成部分积，并利用多级 CSA 压缩树将多路部分积逐步压缩为两路，最后通过一次普通加法得到乘积。由于其实现方式不同于迭代乘法，因此这里不再使用“理论最长周期”来直接衡量，而是在本次实验的三组仿真样例中统计得到其周期数均为 3。与基础优化版相比，华莱士树优化版在样例测试中的周期进一步减少，说明其性能更优。

需要说明的是，表中的“理论最长周期”反映的是算法在最坏情况下的性能上界，而“仿真样例周期”则是针对所选三组测试数据得到的实验统计结果。两者含义不同，但都能够反映乘法器性能：前者体现算法本身的理论效率，后者体现具体测试样例下的实际执行效果。

综合来看，基础优化版的优点是修改量较小，保留了指导书参考设计的大部分结构，容易理解和实现；华莱士树优化版的优点是性能更高，能够进一步减少乘法运算所需周期，但实现复杂度也更高。三种方案都能够正确完成 32 位有符号乘法运算，其中华莱士树优化版的性能最好。

8 实验收获

通过本次实验，我进一步理解了定点乘法器的实现原理，尤其是有符号补码乘法中“先取绝对值，再恢复符号”的处理方法。相比单纯理解公式，这次通过自己修改代码、观察波形和上板验证，对乘

法器内部的数据流动和结果形成过程有了更直观的认识。

在基础优化版的实现过程中，我体会到性能优化并不一定需要完全推翻原有结构，有时只要抓住关键路径进行改进，就能够取得较明显的效果。本实验中，通过把每拍处理位数从 1 位提高到 2 位，就把乘法器的理论最长周期从 32 降低到了 16，同时也保持了原有设计思路的清晰性。

在进一步实现华莱士树优化版时，我对并行部分积生成、CSA 压缩树以及最终加法器的作用有了初步认识。相比基础优化版，华莱士树版实现起来更复杂，但也让我认识到不同算法在硬件设计中的性能差异。通过这两个版本的对比，我对“结构决定性能”这一点有了更深的理解。

此外，本次实验也让我更加熟悉 Vivado 的使用流程，包括模块修改、testbench 编写、行为级仿真、添加顶层模块、约束文件配置以及 FPGA 实验箱上板验证等内容。通过完成从代码到仿真、再到硬件验证的完整流程，我对后续组成原理实验和 CPU 相关实验的实现过程也有了更强的信心。

附录 A 附录

基础优化版顶层模块 multiply_display.v

Listing 1: Basic Optimized Top Module: multiply_display.v

```
//************************************************************************
//   > 文件名: multiply_display.v
//   > 描述   : 乘法器显示模块，调用FPGA板上的IO接口和触摸屏
//   > 作者   : LOONGSON
//   > 日期   : 2016-04-14
//************************************************************************
module multiply_display(
    //时钟与复位信号
    input clk,
    input resetn,    //后缀"n"代表低电平有效

    //拨码开关，用于选择输入数
    input input_sel, //0:输入为乘数1;1:输入为乘数2
    input sw_begin,

    //乘法结束信号
    output led_end,

    //触摸屏相关接口，不需要更改
    output lcd_rst,
    output lcd_cs,
    output lcd_rs,
    output lcd_wr,
    output lcd_rd,
    inout[15:0] lcd_data_io,
    output lcd_bl_ctr,
    inout ct_int,
    inout ct_sda,
    output ct_scl,
    output ct_rstn
);
//-----{调用乘法器模块}begin
    wire        mult_begin;
    reg  [31:0] mult_op1;
    reg  [31:0] mult_op2;
    wire [63:0] product;
    wire        mult_end;
    assign mult_begin = sw_begin;
    assign led_end = mult_end;
    multiply_wallace multiply_module (
        .clk       (clk       ),
        .mult_begin(mult_begin),
        .mult_op1  (mult_op1  ),
        .mult_op2  (mult_op2  ),
        .product   (product   ),
        .mult_end  (mult_end  )
    );
    reg [63:0] product_r;
    always @(posedge clk)
    begin
```

```
51          if (!resetn)
52          begin
53              product_r <= 64'd0;
54          end
55          else if (mult_end)
56          begin
57              product_r <= product;
58          end
59      end
60  //-----{调用乘法器模块}end
61
62  //--------------------{调用触摸屏模块}begin--------------------//
63  //-----{实例化触摸屏}begin
64  //此小节不需要更改
65      reg         display_valid;
66      reg  [39:0] display_name;
67      reg  [31:0] display_value;
68      wire [5 :0] display_number;
69      wire        input_valid;
70      wire [31:0] input_value;
71
72      lcd_module lcd_module(
73          .clk            (clk           ),   //10Mhz
74          .resetn         (resetn        ),
75
76          //调用触摸屏的接口
77          .display_valid  (display_valid ),
78          .display_name   (display_name  ),
79          .display_value  (display_value ),
80          .display_number (display_number),
81          .input_valid    (input_valid   ),
82          .input_value    (input_value   ),
83
84          //lcd触摸屏相关接口，不需要更改
85          .lcd_rst        (lcd_rst       ),
86          .lcd_cs         (lcd_cs        ),
87          .lcd_rs         (lcd_rs        ),
88          .lcd_wr         (lcd_wr        ),
89          .lcd_rd         (lcd_rd        ),
90          .lcd_data_io    (lcd_data_io   ),
91          .lcd_bl_ctr     (lcd_bl_ctr    ),
92          .ct_int         (ct_int        ),
93          .ct_sda         (ct_sda        ),
94          .ct_scl         (ct_scl        ),
95          .ct_rstn        (ct_rstn       )
96      );
97  //-----{实例化触摸屏}end
98
99  //-----{从触摸屏获取输入}begin
100 //根据实际需要输入的数修改此小节，
101 //建议对每一个数的输入，编写单独一个always块
102     //当input_sel为0时，表示输入数为乘数1
103     always @(posedge clk)
104     begin
105         if (!resetn)
106         begin
107             mult_op1 <= 32'd0;
108         end
109         else if (input_valid && !input_sel)
110         begin
111             mult_op1 <= input_value;
112         end
113     end
114
115     //当input_sel为1时，表示输入数为乘数2
116     always @(posedge clk)
117     begin
118         if (!resetn)
119         begin
120             mult_op2 <= 32'd0;
121         end
122         else if (input_valid && input_sel)
123         begin
124             mult_op2 <= input_value;
125         end
126     end
127 //-----{从触摸屏获取输入}end
128
129 //-----{输出到触摸屏显示}begin
130 //根据需要显示的数修改此小节，
```

```
131 //触摸屏上共有44块显示区域，可显示44组32位数据
132 //44块显示区域从1开始编号，编号为1-44，
133     always @(posedge clk)
134     begin
135         case(display_number)
136             6'd1 :
137             begin
138                 display_valid <= 1'b1;
139                 display_name  <= "M_OP1";
140                 display_value <= mult_op1;
141             end
142             6'd2 :
143             begin
144                 display_valid <= 1'b1;
145                 display_name  <= "M_OP2";
146                 display_value <= mult_op2;
147             end
148             6'd3 :
149             begin
150                 display_valid <= 1'b1;
151                 display_name  <= "PRO_H";
152                 display_value <= product_r[63:32];
153             end
154             6'd4 :
155             begin
156                 display_valid <= 1'b1;
157                 display_name  <= "PRO_L";
158                 display_value <= product_r[31: 0];
159             end
160             default :
161             begin
162                 display_valid <= 1'b0;
163                 display_name  <= 48'd0;
164                 display_value <= 32'd0;
165             end
166         endcase
167     end
168 //-----{输出到触摸屏显示}end
169 //----------------------{调用触摸屏模块}end--------------------//
170 endmodule
```

基础优化版乘法器模块 multiply.v

Listing 2: Basic Optimized Multiplier Module: multiply.v

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////
3  **
4  // > 文件名: multiply.v
5  // > 描述 : 乘法器模块，低效率的迭代乘法算法，使用两个乘数绝对值参与运算
6  // > 作者 : LOONGSON
7  // > 日期 : 2016-04-14
8  //////////////////////////////////////////////////////////////////
9  **
10
11 module multiply( // 乘法器
12 input clk,                // 时钟
13 input mult_begin,         // 乘法开始信号
14 input [31:0] mult_op1,    // 乘法源操作数1
15 input [31:0] mult_op2,    // 乘法源操作数2
16 output [63:0] product,    // 乘积
17 output mult_end           // 乘法结束信号
18 );
19
20 //乘法正在运算信号和结束信号
21 reg mult_valid;
22
23 //结束条件仍然使用"乘数移空"为结束标志
24 assign mult_end = mult_valid & ~(|multiplier); //乘法结束信号：乘数全0
25
26 always @(posedge clk)
27 begin
28     if (!mult_begin || mult_end)
29     begin
30         mult_valid <= 1'b0;
31     end
32     else
33     begin
```

```
        mult_valid <= 1'b1;
    end
end

//两个源操作取绝对值，正数的绝对值为其本身，负数的绝对值为取反加1
wire op1_sign;            //操作数1的符号位
wire op2_sign;            //操作数2的符号位
wire [31:0] op1_absolute; //操作数1的绝对值
wire [31:0] op2_absolute; //操作数2的绝对值
assign op1_sign = mult_op1[31];
assign op2_sign = mult_op2[31];
assign op1_absolute = op1_sign ? (~mult_op1+1) : mult_op1;
assign op2_absolute = op2_sign ? (~mult_op2+1) : mult_op2;

//加载被乘数，运算时每次左移一位
reg [63:0] multiplicand;
always @ (posedge clk)
begin
    if (mult_valid)
    begin
        //【修改】手册原版为每次左移1位：
        // multiplicand <= {multiplicand[62:0],1'b0};

        //【修改】2位乘优化：每次左移2位
        multiplicand <= {multiplicand[61:0],2'b00};
    end
    else if (mult_begin)
    begin
        //乘法开始时装载被乘数绝对值
        multiplicand <= {32'd0,op1_absolute};
    end
end

//加载乘数，运算时每次右移一位
reg [31:0] multiplier;
always @ (posedge clk)
begin
    if (mult_valid)
    begin
        //【修改】手册原版为每次右移1位：
        // multiplier <= {1'b0,multiplier[31:1]};

        //【修改后】2位乘优化：每次右移2位
        multiplier <= {2'b00,multiplier[31:2]};
    end
    else if (mult_begin)
    begin
        //乘法开始时装载乘数绝对值
        multiplier <= op2_absolute;
    end
end

// 部分积：乘数末位为1，由被乘数左移得到；乘数末位为0，部分积为0
// wire [63:0] partial_product;
// assign partial_product = multiplier[0] ? multiplicand : 64'd0;

//【删除】以上是手册原版"1位乘"的部分积生成方式，只判断 multiplier[0]
//【新增】改为"2位乘"方式，根据 multiplier[1:0] 生成 0、M、2M、3M 四种部分积
reg [63:0] partial_product;
always @(*)
begin
    case (multiplier[1:0])
        2'b00: partial_product = 64'd0;                          //【新增】加0
        2'b01: partial_product = multiplicand;                   //【新增】加1倍M
        2'b10: partial_product = (multiplicand << 1);            //【新增】加2倍M
        2'b11: partial_product = multiplicand + (multiplicand << 1); //【新增】加3倍M
        default: partial_product = 64'd0;
    endcase
end

//累加器
reg [63:0] product_temp;
always @ (posedge clk)
begin
    if (mult_valid)
    begin
        //继续累加当前部分积
        product_temp <= product_temp + partial_product;
    end
    else if (mult_begin)
```

```
    begin
        //乘法开始，乘积清零
        product_temp <= 64'd0;
    end
end

//乘法结果的符号位和乘法结果
reg product_sign;
always @ (posedge clk) // 乘积符号
begin
    if (mult_valid)
    begin
        //结果符号仍为两操作数符号异或
        product_sign <= op1_sign ^ op2_sign;
    end
end

//若乘法结果为负数，则需要对结果取反+1
assign product = product_sign ? (~product_temp+1) : product_temp;

endmodule
```

基础优化版仿真文件 testbench.v

Listing 3: Basic Optimized Testbench: testbench.v

```
`timescale 1ns / 1ps

module testbench;

    // Inputs
    reg         clk;
    reg         mult_begin;
    reg [31:0]  mult_op1;
    reg [31:0]  mult_op2;

    // Outputs
    wire [63:0] product;
    wire        mult_end;

    // 用于统计周期数
    integer cycle_cnt;

    // Instantiate the Unit Under Test (UUT)
    multiply uut (
        .clk       (clk),
        .mult_begin(mult_begin),
        .mult_op1  (mult_op1),
        .mult_op2  (mult_op2),
        .product   (product),
        .mult_end  (mult_end)
    );

    // 10ns 时钟周期
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // 单组测试任务
    task run_case;
        input signed [31:0] a;
        input signed [31:0] b;
        reg   signed [63:0] expect;
        begin
            expect = a * b;
            cycle_cnt = 0;

            mult_op1   = a;
            mult_op2   = b;
            mult_begin = 1'b1;

            while (!mult_end) begin
                @(posedge clk);
                cycle_cnt = cycle_cnt + 1;
            end

            @(posedge clk);
```

```
            mult_begin = 1'b0;

            $display("-----------------------------------------------");
            $display("a       = %0d, b = %0d", a, b);
            $display("expect  = %0d, hex = 0x%h", expect, expect);
            $display("product = %0d, hex = 0x%h", $signed(product), product);
            $display("cycles  = %0d", cycle_cnt);

            if ($signed(product) !== expect) begin
                $display("ERROR: result mismatch!");
            end
            else begin
                $display("PASS");
            end

            #20;
        end
    endtask

    initial begin

        mult_begin = 1'b0;
        mult_op1   = 32'd0;
        mult_op2   = 32'd0;

        #20;

//        第一次
          run_case(32'sd13, 32'sd11);
//        第二次
//        run_case(-32'sd13, 32'sd11);
//        第三次
//        run_case(-32'sd25, -32'sd4);

        #100;
        $finish;
    end

endmodule
```

华莱士树版顶层模块 multiply_display.v

Listing 4: Wallace Tree Top Module: multiply_display.v

```
//*************************************************************************
//   > 文件名: multiply_display.v
//   > 描述   : 乘法器显示模块，调用FPGA板上的IO接口和触摸屏
//   > 作者   : LOONGSON
//   > 日期   : 2016-04-14
//*************************************************************************
module multiply_display(
    //时钟与复位信号
    input clk,
    input resetn,    //后缀"n"代表低电平有效

    //拨码开关，用于选择输入数
    input input_sel, //0:输入为乘数1;1:输入为乘数2
    input sw_begin,

    //乘法结束信号
    output led_end,

    //触摸屏相关接口，不需要更改
    output lcd_rst,
    output lcd_cs,
    output lcd_rs,
    output lcd_wr,
    output lcd_rd,
    inout[15:0] lcd_data_io,
    output lcd_bl_ctr,
    inout ct_int,
    inout ct_sda,
    output ct_scl,
    output ct_rstn
);
//-----{调用乘法器模块}begin
    wire        mult_begin;
    reg  [31:0] mult_op1;
```

```
35      reg  [31:0] mult_op2;
36      wire [63:0] product;
37      wire        mult_end;
38      assign mult_begin = sw_begin;
39      assign led_end = mult_end;
40      multiply_wallace multiply_module (
41          .clk       (clk       ),
42          .mult_begin(mult_begin),
43          .mult_op1  (mult_op1  ),
44          .mult_op2  (mult_op2  ),
45          .product   (product   ),
46          .mult_end  (mult_end  )
47      );
48      reg [63:0] product_r;
49      always @(posedge clk)
50      begin
51          if (!resetn)
52          begin
53              product_r <= 64'd0;
54          end
55          else if (mult_end)
56          begin
57              product_r <= product;
58          end
59      end
60  //-----{调用乘法器模块}end
61
62  //--------------------{调用触摸屏模块}begin--------------------//
63  //-----{实例化触摸屏}begin
64  //此小节不需要更改
65      reg         display_valid;
66      reg  [39:0] display_name;
67      reg  [31:0] display_value;
68      wire [5 :0] display_number;
69      wire        input_valid;
70      wire [31:0] input_value;
71
72      lcd_module lcd_module(
73          .clk            (clk           ),   //10Mhz
74          .resetn         (resetn        ),
75
76          //调用触摸屏的接口
77          .display_valid  (display_valid ),
78          .display_name   (display_name  ),
79          .display_value  (display_value ),
80          .display_number (display_number),
81          .input_valid    (input_valid   ),
82          .input_value    (input_value   ),
83
84          //lcd触摸屏相关接口，不需要更改
85          .lcd_rst        (lcd_rst       ),
86          .lcd_cs         (lcd_cs        ),
87          .lcd_rs         (lcd_rs        ),
88          .lcd_wr         (lcd_wr        ),
89          .lcd_rd         (lcd_rd        ),
90          .lcd_data_io    (lcd_data_io   ),
91          .lcd_bl_ctr     (lcd_bl_ctr    ),
92          .ct_int         (ct_int        ),
93          .ct_sda         (ct_sda        ),
94          .ct_scl         (ct_scl        ),
95          .ct_rstn        (ct_rstn       )
96      );
97  //-----{实例化触摸屏}end
98
99  //-----{从触摸屏获取输入}begin
100 //根据实际需要输入的数修改此小节，
101 //建议对每一个数的输入，编写单独一个always块
102     //当input_sel为0时，表示输入数为乘数1
103     always @(posedge clk)
104     begin
105         if (!resetn)
106         begin
107             mult_op1 <= 32'd0;
108         end
109         else if (input_valid && !input_sel)
110         begin
111             mult_op1 <= input_value;
112         end
113     end
114
```

```
115     //当input_sel为1时，表示输入数为乘数2
116     always @(posedge clk)
117     begin
118         if (!resetn)
119         begin
120             mult_op2 <= 32'd0;
121         end
122         else if (input_valid && input_sel)
123         begin
124             mult_op2 <= input_value;
125         end
126     end
127 //-----{从触摸屏获取输入}end
128 
129 //-----{输出到触摸屏显示}begin
130 //根据需要显示的数修改此小节，
131 //触摸屏上共有44块显示区域，可显示44组32位数据
132 //44块显示区域从1开始编号，编号为1-44，
133     always @(posedge clk)
134     begin
135         case(display_number)
136             6'd1 :
137             begin
138                 display_valid <= 1'b1;
139                 display_name  <= "M_OP1";
140                 display_value <= mult_op1;
141             end
142             6'd2 :
143             begin
144                 display_valid <= 1'b1;
145                 display_name  <= "M_OP2";
146                 display_value <= mult_op2;
147             end
148             6'd3 :
149             begin
150                 display_valid <= 1'b1;
151                 display_name  <= "PRO_H";
152                 display_value <= product_r[63:32];
153             end
154             6'd4 :
155             begin
156                 display_valid <= 1'b1;
157                 display_name  <= "PRO_L";
158                 display_value <= product_r[31: 0];
159             end
160             default :
161             begin
162                 display_valid <= 1'b0;
163                 display_name  <= 48'd0;
164                 display_value <= 32'd0;
165             end
166         endcase
167     end
168 //-----{输出到触摸屏显示}end
169 //----------------------{调用触摸屏模块}end--------------------//
170 endmodule
```

华莱士树版乘法器模块 multiply_wallace.v

Listing 5: Wallace Tree Multiplier Module: multiply_wallace.v

```
1  `timescale 1ns / 1ps
2  
3  //************************************************************
4  // > 文件名: multiply_wallace.v
5  // > 描述   : 华莱士树 (压缩树) 优化版乘法器
6  // > 说明   : 尽量保留手册参考 multiply.v 的接口风格
7  // >          主要变化为:
8  // >          1. 不再采用逐拍移位累加的迭代乘法
9  // >          2. 改为并行生成部分积
10 // >          3. 使用 CSA 压缩树 (Wallace Tree 风格) 进行多操作数压缩
11 // >          4. 最终使用一次加法得到无符号绝对值乘积
12 // >          5. 再进行符号恢复，得到补码乘积
13 //************************************************************
14 module multiply_wallace(
15     input         clk,         // 时钟
16     input         mult_begin,  // 乘法开始信号
17     input  [31:0] mult_op1,    // 乘法源操作数1 (补码)
```

```
18      input  [31:0] mult_op2,    // 乘法源操作数2（补码）
19      output [63:0] product,     // 乘积（补码）
20      output        mult_end     // 乘法结束信号
21  );
22
23      //============================================================
24      // 1) 运算控制
25      //============================================================
26      //【修改】手册原版是"迭代乘法"，mult_end 由乘数移空决定
27      //【修改】华莱士树核心为并行组合逻辑，为了兼容原显示模块，
28      //         这里把结果先寄存，再将 mult_end 延后一拍输出
29      reg mult_valid;
30      reg mult_end_r;
31
32      assign mult_end = mult_end_r;
33
34      always @(posedge clk) begin
35          if (!mult_begin) begin
36              mult_valid <= 1'b0;
37              mult_end_r <= 1'b0;
38          end
39          else begin
40              mult_valid <= 1'b1;       //【新增】表示本次运算已经进入有效阶段
41              mult_end_r <= mult_valid; //【新增】延后一拍输出结束信号，方便外围锁存结果
42          end
43      end
44
45      //============================================================
46      // 2) 两个源操作数先取绝对值
47      //============================================================
48      //【保留手册思路】仍然采用"先取绝对值参与运算，最后恢复符号"的方式
49      wire        op1_sign;
50      wire        op2_sign;
51      wire [31:0] op1_absolute;
52      wire [31:0] op2_absolute;
53
54      assign op1_sign     = mult_op1[31];
55      assign op2_sign     = mult_op2[31];
56      assign op1_absolute = op1_sign ? (~mult_op1 + 32'd1) : mult_op1;
57      assign op2_absolute = op2_sign ? (~mult_op2 + 32'd1) : mult_op2;
58
59      //============================================================
60      // 3) CSA 压缩器函数
61      //============================================================
62      //【新增】华莱士树核心：把3个操作数压缩成2个操作数
63      // sum   = a ^ b ^ c
64      // carry = majority(a,b,c) << 1
65      function [63:0] csa_sum;
66          input [63:0] a;
67          input [63:0] b;
68          input [63:0] c;
69          begin
70              csa_sum = a ^ b ^ c;
71          end
72      endfunction
73
74      function [63:0] csa_carry;
75          input [63:0] a;
76          input [63:0] b;
77          input [63:0] c;
78          begin
79              csa_carry = ((a & b) | (a & c) | (b & c)) << 1;
80          end
81      endfunction
82
83      //============================================================
84      // 4) 部分积与压缩树中间结果
85      //============================================================
86      //【新增】并行部分积
87      reg [63:0] pp [0:31];
88
89      //【新增】各级压缩树中间结果
90      reg [63:0] stage1 [0:21]; // 32 -> 22
91      reg [63:0] stage2 [0:14]; // 22 -> 15
92      reg [63:0] stage3 [0:9 ]; // 15 -> 10
93      reg [63:0] stage4 [0:6 ]; // 10 -> 7
94      reg [63:0] stage5 [0:4 ]; // 7  -> 5
95      reg [63:0] stage6 [0:3 ]; // 5  -> 4
96      reg [63:0] stage7 [0:2 ]; // 4  -> 3
97
```

```
98      reg [63:0] wallace_a;
99      reg [63:0] wallace_b;
100     reg [63:0] unsigned_product;
101
102     integer i;
103
104     //=======================================================
105     // 5) 华莱士树（压缩树）组合逻辑
106     //=======================================================
107     always @(*) begin
108         //-------------------------------------------------------
109         // 5.1 并行生成 32 个部分积
110         //-------------------------------------------------------
111         //【新增】与迭代乘法不同，这里不再逐拍移位相加，
112         //       而是一次性生成全部部分积
113         for (i = 0; i < 32; i = i + 1) begin
114             if (op2_absolute[i]) begin
115                 pp[i] = ({32'd0, op1_absolute} << i);
116             end
117             else begin
118                 pp[i] = 64'd0;
119             end
120         end
121
122         //-------------------------------------------------------
123         // 5.2 stage1 : 32 -> 22
124         //-------------------------------------------------------
125         stage1[ 0] = csa_sum  (pp[ 0], pp[ 1], pp[ 2]);
126         stage1[ 1] = csa_carry(pp[ 0], pp[ 1], pp[ 2]);
127         stage1[ 2] = csa_sum  (pp[ 3], pp[ 4], pp[ 5]);
128         stage1[ 3] = csa_carry(pp[ 3], pp[ 4], pp[ 5]);
129         stage1[ 4] = csa_sum  (pp[ 6], pp[ 7], pp[ 8]);
130         stage1[ 5] = csa_carry(pp[ 6], pp[ 7], pp[ 8]);
131         stage1[ 6] = csa_sum  (pp[ 9], pp[10], pp[11]);
132         stage1[ 7] = csa_carry(pp[ 9], pp[10], pp[11]);
133         stage1[ 8] = csa_sum  (pp[12], pp[13], pp[14]);
134         stage1[ 9] = csa_carry(pp[12], pp[13], pp[14]);
135         stage1[10] = csa_sum  (pp[15], pp[16], pp[17]);
136         stage1[11] = csa_carry(pp[15], pp[16], pp[17]);
137         stage1[12] = csa_sum  (pp[18], pp[19], pp[20]);
138         stage1[13] = csa_carry(pp[18], pp[19], pp[20]);
139         stage1[14] = csa_sum  (pp[21], pp[22], pp[23]);
140         stage1[15] = csa_carry(pp[21], pp[22], pp[23]);
141         stage1[16] = csa_sum  (pp[24], pp[25], pp[26]);
142         stage1[17] = csa_carry(pp[24], pp[25], pp[26]);
143         stage1[18] = csa_sum  (pp[27], pp[28], pp[29]);
144         stage1[19] = csa_carry(pp[27], pp[28], pp[29]);
145         stage1[20] = pp[30];
146         stage1[21] = pp[31];
147
148         //-------------------------------------------------------
149         // 5.3 stage2 : 22 -> 15
150         //-------------------------------------------------------
151         stage2[ 0] = csa_sum  (stage1[ 0], stage1[ 1], stage1[ 2]);
152         stage2[ 1] = csa_carry(stage1[ 0], stage1[ 1], stage1[ 2]);
153         stage2[ 2] = csa_sum  (stage1[ 3], stage1[ 4], stage1[ 5]);
154         stage2[ 3] = csa_carry(stage1[ 3], stage1[ 4], stage1[ 5]);
155         stage2[ 4] = csa_sum  (stage1[ 6], stage1[ 7], stage1[ 8]);
156         stage2[ 5] = csa_carry(stage1[ 6], stage1[ 7], stage1[ 8]);
157         stage2[ 6] = csa_sum  (stage1[ 9], stage1[10], stage1[11]);
158         stage2[ 7] = csa_carry(stage1[ 9], stage1[10], stage1[11]);
159         stage2[ 8] = csa_sum  (stage1[12], stage1[13], stage1[14]);
160         stage2[ 9] = csa_carry(stage1[12], stage1[13], stage1[14]);
161         stage2[10] = csa_sum  (stage1[15], stage1[16], stage1[17]);
162         stage2[11] = csa_carry(stage1[15], stage1[16], stage1[17]);
163         stage2[12] = csa_sum  (stage1[18], stage1[19], stage1[20]);
164         stage2[13] = csa_carry(stage1[18], stage1[19], stage1[20]);
165         stage2[14] = stage1[21];
166
167         //-------------------------------------------------------
168         // 5.4 stage3 : 15 -> 10
169         //-------------------------------------------------------
170         stage3[0] = csa_sum  (stage2[ 0], stage2[ 1], stage2[ 2]);
171         stage3[1] = csa_carry(stage2[ 0], stage2[ 1], stage2[ 2]);
172         stage3[2] = csa_sum  (stage2[ 3], stage2[ 4], stage2[ 5]);
173         stage3[3] = csa_carry(stage2[ 3], stage2[ 4], stage2[ 5]);
174         stage3[4] = csa_sum  (stage2[ 6], stage2[ 7], stage2[ 8]);
175         stage3[5] = csa_carry(stage2[ 6], stage2[ 7], stage2[ 8]);
176         stage3[6] = csa_sum  (stage2[ 9], stage2[10], stage2[11]);
177         stage3[7] = csa_carry(stage2[ 9], stage2[10], stage2[11]);
```

```
        stage3[8] = csa_sum  (stage2[12], stage2[13], stage2[14]);
        stage3[9] = csa_carry(stage2[12], stage2[13], stage2[14]);

        //------------------------------------------------------
        // 5.5 stage4 : 10 -> 7
        //------------------------------------------------------
        stage4[0] = csa_sum  (stage3[0], stage3[1], stage3[2]);
        stage4[1] = csa_carry(stage3[0], stage3[1], stage3[2]);
        stage4[2] = csa_sum  (stage3[3], stage3[4], stage3[5]);
        stage4[3] = csa_carry(stage3[3], stage3[4], stage3[5]);
        stage4[4] = csa_sum  (stage3[6], stage3[7], stage3[8]);
        stage4[5] = csa_carry(stage3[6], stage3[7], stage3[8]);
        stage4[6] = stage3[9];

        //------------------------------------------------------
        // 5.6 stage5 : 7 -> 5
        //------------------------------------------------------
        stage5[0] = csa_sum  (stage4[0], stage4[1], stage4[2]);
        stage5[1] = csa_carry(stage4[0], stage4[1], stage4[2]);
        stage5[2] = csa_sum  (stage4[3], stage4[4], stage4[5]);
        stage5[3] = csa_carry(stage4[3], stage4[4], stage4[5]);
        stage5[4] = stage4[6];

        //------------------------------------------------------
        // 5.7 stage6 : 5 -> 4
        //------------------------------------------------------
        stage6[0] = csa_sum  (stage5[0], stage5[1], stage5[2]);
        stage6[1] = csa_carry(stage5[0], stage5[1], stage5[2]);
        stage6[2] = stage5[3];
        stage6[3] = stage5[4];

        //------------------------------------------------------
        // 5.8 stage7 : 4 -> 3
        //------------------------------------------------------
        stage7[0] = csa_sum  (stage6[0], stage6[1], stage6[2]);
        stage7[1] = csa_carry(stage6[0], stage6[1], stage6[2]);
        stage7[2] = stage6[3];

        //------------------------------------------------------
        // 5.9 最后一轮压缩: 3 -> 2, 再做一次普通加法
        //------------------------------------------------------
        wallace_a = csa_sum  (stage7[0], stage7[1], stage7[2]);
        wallace_b = csa_carry(stage7[0], stage7[1], stage7[2]);

        //【新增】最终进位传播加法器
        unsigned_product = wallace_a + wallace_b;
    end

    //============================================================
    // 6) 寄存无符号乘积, 并恢复符号
    //============================================================
    reg [63:0] product_temp;
    reg        product_sign;

    always @(posedge clk) begin
        if (mult_begin) begin
            //【新增】锁存华莱士树压缩后的无符号乘积
            product_temp <= unsigned_product;

            //结果符号 = 两输入符号异或
            product_sign <= op1_sign ^ op2_sign;
        end
    end

    //若结果应为负, 则取反加1恢复补码
    assign product = product_sign ? (~product_temp + 64'd1) : product_temp;

endmodule
```

华莱士树版仿真文件 testbench.v

Listing 6: Wallace Tree Testbench: testbench.v

```
`timescale 1ns / 1ps

module testbench;

    // Inputs
```

```
    reg         clk;
    reg         mult_begin;
    reg [31:0]  mult_op1;
    reg [31:0]  mult_op2;

    // Outputs
    wire [63:0] product;
    wire        mult_end;

    // 用于统计周期数
    integer cycle_cnt;

    // Instantiate the Unit Under Test (UUT)
    multiply_wallace uut (
        .clk       (clk),
        .mult_begin(mult_begin),
        .mult_op1  (mult_op1),
        .mult_op2  (mult_op2),
        .product   (product),
        .mult_end  (mult_end)
    );

    // 10ns 时钟周期
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // 单组测试任务
    task run_case;
        input signed [31:0] a;
        input signed [31:0] b;
        reg   signed [63:0] expect;
        begin
            expect = a * b;
            cycle_cnt = 0;

            mult_op1   = a;
            mult_op2   = b;
            mult_begin = 1'b1;

            while (!mult_end) begin
                @(posedge clk);
                cycle_cnt = cycle_cnt + 1;
            end

            @(posedge clk);
            mult_begin = 1'b0;

            $display("-------------------------------------------------");
            $display("a       = %0d, b = %0d", a, b);
            $display("expect  = %0d, hex = 0x%h", expect, expect);
            $display("product = %0d, hex = 0x%h", $signed(product), product);
            $display("cycles  = %0d", cycle_cnt);

            if ($signed(product) !== expect) begin
                $display("ERROR: result mismatch!");
            end
            else begin
                $display("PASS");
            end

            #20;
        end
    endtask

    initial begin

        mult_begin = 1'b0;
        mult_op1   = 32'd0;
        mult_op2   = 32'd0;

        #20;

//        第一次
          run_case(32'sd13, 32'sd11);
//        第二次
//        run_case(-32'sd13, 32'sd11);
//        第三次
//        run_case(-32'sd25, -32'sd4);
```

```
86
87          #100;
88          $finish;
89      end
90
91  endmodule
```